

Uç Bilişimde Hibrit Zeka: TinyML ve SLM İşbirliği ile Otonom Endüstriyel IoT Yönetimi

Mehmet Salih SİNCAR

Özet

Nesnelerin İnterneti (IoT) ve Endüstri 4.0 sistemleri, veri işleme, hata tespiti ve karar alma süreçlerinde yoğun bir şekilde merkezi bulut sunucularına bağımlı olarak çalışmaktadırlar. Ancak, siber saldırı tehditlerinin artması, internet bağlantısının kopması durumunda kritik altyapıların tamamen işlevsiz kalması ve bulut sunucularına veri aktarmanın oluşturmuş olduğu yüksek gecikme süreleri endüstriyel operasyonları tehdit etmektedir. Son zamanlarda, Küçük Dil Modellerinin (SLM) cihaz üzerinde kullanılabilmesi bu bağımlılığı kırma potansiyeli taşısa da; kısıtlı hafıza, düşük işlemci gücü ve enerji

Bu çalışmada, kısıtlı donanımlara sahip uç bilişim cihazlarında yüksek performansla çalışabilen, “Sıfır Güven (Zero Trust)” ve “Hibrit Zeka” tabanlı yeni bir otonom endüstriyel IoT mimarisi önerilmektedir. Geliştirilen sistem; deterministik donanım kontrolünü sağlayan GoLang tabanlı bir orkestratör, hafif ve sürekli aktif bir yapay zeka olan TinyML (Autoencoder) ve yalnızca kritik anlarda uyanan yapılandırılmış bir SLM (Gemma:2b) olmak üzere üç katmanlı bir Docker konteyner mimarisinden oluşmaktadır. Sensör verileri, Fiziksel Klonlanamaz Fonksiyon (PUF) ile donanım düzeyinde doğrulanmakta ve biyo-esinlenilmiş adaptif bir algoritma ile dinamik olarak örneklenmektedir. TinyML modeli tarafından tespit edilen olası bir anomali durumunda, sistem otonom olarak SLM’i tetiklemekte ve arızanın kök nedenini analiz ettirerek JSON formatında acil durum makine komutları üretmektedir. Üretilen komutlar, orkestratörün öncelikli işlem mekanizması ile sıfır gecikmeyle işlenerek fiziksel operasyonları güvenli bir şekilde askıya almakta ve arıza durumunu denetim mekanizmaları için kayıt altına almaktadır. Elde edilen sonuçlar; önerilen hiyerarşik mimarinin, sürekli çalışan standart modellere kıyasla uç cihazlarda enerji verimliliğini maksimize ettiğini, siber saldırılara karşı donanım seviyesinde güvenlik sağladığını ve internet bağlantısı olmadan da Endüstri 5.0 hedeflerine uygun tam otonom karar alma yeteneği sunduğunu doğrulamaktadır.

Anahtar Kelimeler: Uç Bilişim (Edge Computing), Küçük Dil Modelleri (SLM), TinyML, Hibrit Zeka, Sıfır Güven Mimarisi (Zero Trust), Fiziksel Klonlanamaz Fonksiyon (PUF).

Giriş

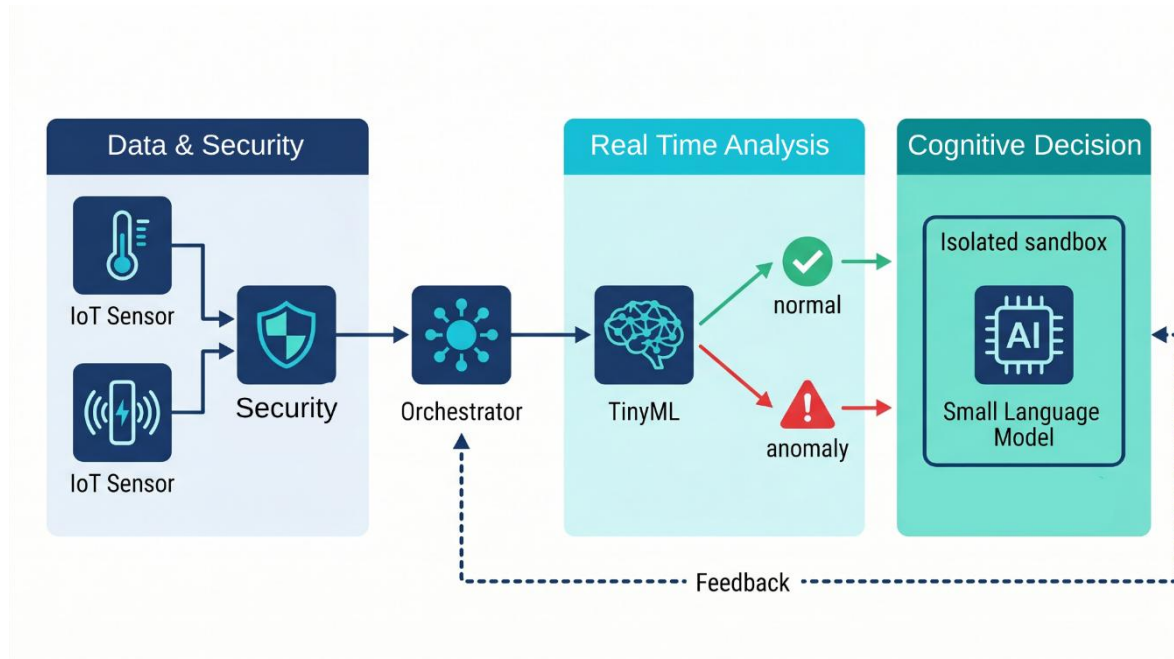
IoT ve Endüstri 4.0 uygulamaları, operasyonel verimliliği artırmak ve büyük verileri analiz etmek için geleneksel olarak merkezi bulut bilişim altyapılarına dayanmaktadır; ancak artan veri hacmi ve siber güvenlik tehditleri, buluta olan bu bağımlılığın getirdiği gecikme ve kesinti sorunlarını kritik hale getirmiştir[8], [24]. Endüstri 5.0 vizyonu ile birlikte, sistemlerin yalnızca veriyi toplayan ve ileten yapılar olmaktan çıkıp, insan müdahalesine gerek kalmadan otonom bir şekilde kararlar olabilen bilişsel yapılara dönüşmesi hedeflenmektedir [1]. Bu durum, yapay zeka algoritmalarının doğrudan veriler üretme noktasında, yani uç cihazlarda çalıştırılmasını ve bulut iletişiminden bağımsız, çevrimdışı otonom mimarilerin geliştirilmesini endüstriyel sistemlerin en temel önceliği haline getirmiştir [13], [23].

Uç bilişim donanımlarının kısıtlı işlemci ve bellek kapasiteleri, makine öğrenimi modellerinin bu cihazlara entegrasyonunda donanım seviyesinde optimizasyonları zorunlu hale getirmiştir. Bu bağlamda, TinyML yaklaşımları, mikro denetleyiciler üzerinde düşük enerji tüketimi ile akustik, titreşim ve sıcaklık gibi anlık sensör verilerindeki anomalileri tespit etmek için yaygın olarak kullanılmaktadır [16], [25]. Literatürde, uç cihazlarda gerçek zamanlı hata tespiti ve kendi kendini onarma mekanizmaları üzerine ve donanım hızlandırıcıları üzerinde çalışan geometrik sınıflandırıcı algoritmalar başarıyla uygulanmıştır [27],[12]. Makine öğreniminin doğruluğu ile donanım verimliliği

arasındaki dengeyi kurmak amacıyla , enerji tüketimini optimize eden pekiştirmeli öğrenme teknikleri ve çevresel faktörlere göre sensör okuma frekansını ayarlayan biyo-esinlenilmiş adaptif algoritmalar uç ağlara entegre edilmiştir. Bununla birlikte, TinyML tabanlı modeller bir arızanın varlığını mikro saniyeler içinde ve yüksek doğrulukla tespit edebilse de, sorunun kök nedenini anlama, bağlamsal akıl yürütme ve operatörlere semantik çözüm önerileri sunma kapasitesinden yoksundur [2], [18], [3].

Sistemlere bu eksik bilişsel akıl yürütme yeteneğini kazandırmak için Büyük Dil Modellerinin (LLM) IoT mimarilerine entegrasyonu giderek artan bir ilgi görmektedir [6]. Ne var ki, geleneksel LLM'lerin milyarlarca parametre içermesi, bu modellerin kaynak kısıtlı IoT cihazlarında yerel olarak çalıştırılmasını pratik olarak imkansız kılmaktadır [7]. Bu darboğazı aşmak amacıyla, daha düşük parametre sayısına sahip SLM'lerin uç cihazlarda kullanımı öne çıkmıştır [21], [22]. SLM'lerin uç cihazlarda bakım asistanı olarak sesli ve metin tabanlı komutlarla kullanımı [9], gerçek zamanlı robotik sistemlerde çıkarım) sürelerinin hızlandırılması ve bulut-uç işbirliği çerçevesinde hata teşhisi için kullanılması üzerine çeşitli mimariler sunulmuştur [17], [10]. Ancak, bir SLM'in kısıtlı bir uç cihazda sürekli aktif kalarak saniyede onlarca sensör verisini aralıksız şekilde analiz etmesi, cihazı termal ve işlemsel bir darboğaza sürüklemektedir.

Bilişsel yeteneklerin uç noktalara taşınması, performans sorunlarının yanı sıra kritik güvenlik zafiyetlerini de beraberinde getirmektedir. İzole bir çalışma ortamı sağlamak için IoT uygulamalarının konteyner mimarileri ve GoLang gibi yüksek performanslı mikro servisler üzerinden çalıştırılması modern sistemlerde sıkça tercih edilmektedir [20], [4]. Ancak bu sistemler dahi uç noktadaki veri zehirlenmesi ve sahte donanım saldırılarına karşı savunmasız kalabilmektedir. Bu riskleri bertaraf etmek için uç bilişimde güvenli hesaplama alanları ve PUF destekli hafif kimlik doğrulama protokolleri geliştirilmiştir [5], [19], [26]. Benzer şekilde, Zero Trust ağlarında veri trafiğini optimize eden Öncelikli Kuyruk mekanizmaları üzerine çalışmalar yapılmıştır [14]. Tüm bu literatürdeki ilerlemelere rağmen, mevcut yaklaşımlar donanım güvenliğini, düşük gecikmeli TinyML anomali tespitini ve SLM tabanlı bilişsel karar alma sistemlerini birbirinden izole parçalar olarak ele almaktadır.



Şekil 1: Önerilen hiyerarşik uç bilişim mimarisi.

Şekilde donanım seviyesinde PUF tabanlı kimlik doğrulaması, TensorFlow Lite ile mikrosaniyelik anomali tespiti ve kriz anlarında otonom olarak devreye giren Gemma:2b SLM'in JSON tabanlı acil durum komut döngüsü blok diyagram şeklinde gösterilmektedir. Kırmızı oklar, kriz tetikleme ve otonom müdahale döngülerini vurgulamaktadır.

Tablo 1: Mevcut Literatürdeki Çalışmaların Önerilen Mimari ile Karşılaştırması

Literatür	Kısıtlı Cihazlarda Çevrimdışı Çalışma	Bilişsel Karar Alma ve Akıl Yürütme	Donanımsal Zero-Trust Güvenliği	Adaptif ve Hibrit Kaynak Yönetimi
[12],[16], [25]	✓	✗	✗	✓
[6], [7], [23].	✗	✓	✗	✗
[8], [9], [17].	✓	✓	✗	✗
[5], [19], [26].	✓	✗	✓	✗
Önerilen Sistem	✓	✓	✓	✓

Bu çalışmada, literatürdeki izole teknolojik çözümlerin kısıtlılıklarını gidermek amacıyla; donanım kısıtlı cihazlarda yüksek enerji verimliliğiyle çalışabilen, bulut bağımsız, "Zero Trust" ve "Hibrit Zeka" tabanlı entegre bir Uç Bilişim mimarisi önerilmektedir. Geliştirilen sistemin literatüre sağladığı temel katkılar şunlardır:

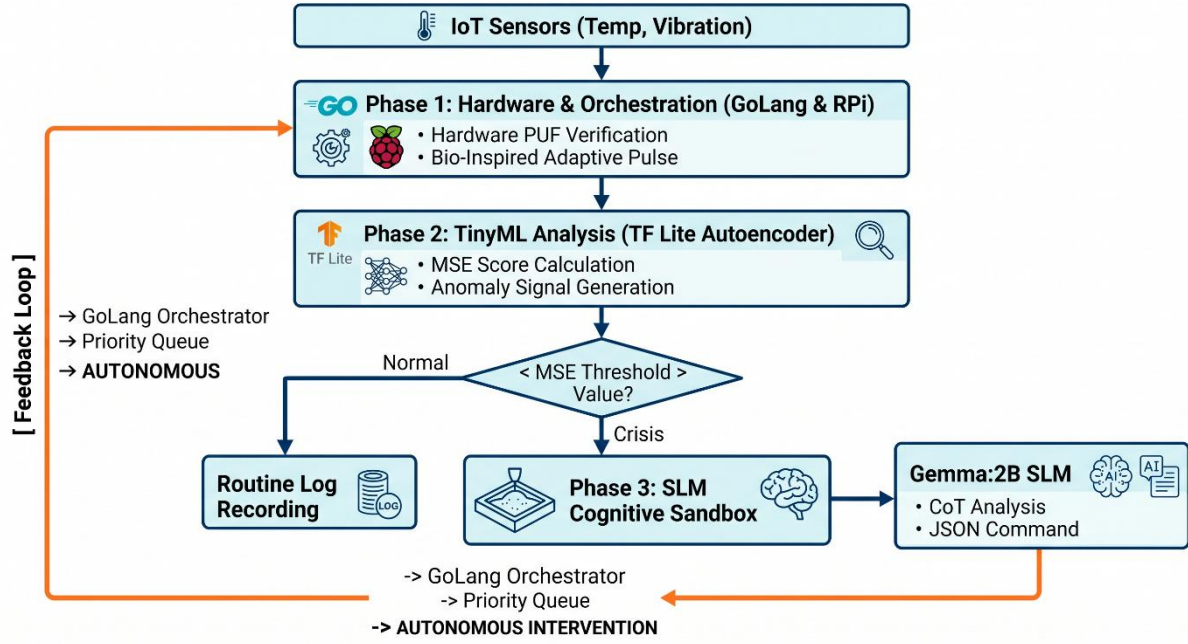
1. Hibrit Yapay Zeka Tasarımı: Kesintisiz veri analizi için hafif bir TinyML modeli kullanılmış, ağır işlem gücü gerektiren SLM yalnızca anomali tespit edildiğinde izole bir "sandbox" içerisinde uyandırılarak uç cihazlardaki termal darboğazlar ve enerji israfı engellenmiştir.
2. Zero Trust ve Donanımsal Koruma: Sisteme entegre edilen PUF tabanlı kimlik doğrulama sayesinde sahte veri akışları engellenmiş; yapay zekanın ürettiği kritik acil durum komutları ise GoLang tabanlı Öncelikli Kuyruk mimarisi ile sıfır gecikmeyle işlenerek sistem güvenli moda alınmıştır.
3. Biyo-Esilenilmiş Kaynak Optimizasyonu: Geliştirilen GoLang orkestratörü, sistemin güncel tehdit ve anomali durumuna göre sensör okuma frekanslarını biyolojik bir nabız gibi dinamik olarak uyarlayarak ağ trafiğini ve işlemci yükünü minimize etmiştir.
4. Çevrimdışı ve Otonom Endüstri 5.0 Altyapısı: Tüm orkestrasyon, otonom çıkarım ve veri denetimi işlemleri Docker konteynerleri ile cihaz üzerinde paketlenerek, ağ bağlantısının tamamen koptuğu durumlarda dahi çalışabilen semantik bir arıza tespit çerçevesi sunulmuştur.

2. Hibrit Zeka ve Sıfır Güven Tabanlı Otonom Uç Mimarisi

Endüstriyel Nesnelerin İnterneti (IIoT) ağlarında anomali tespiti ve otonom kontrol süreçleri, ağırlıklı olarak bulut tabanlı LLM'leri veya izole edilmiş makine öğrenimi sınıflandırıcıları üzerinden yürütülmektedir [6], [13]. Ancak mevcut literatür detaylıca incelendiğinde, bu sistemlerde iyileştirme yapılması gereken kritik sorun alanları olduğu tespit edilmiştir. Öncelikle, LLM'lerin bulut üzerinden çalışması ağ gecikmelerine ve veri gizliliği ihlallerine yol açarken, internet bağlantısının koptuğu durumlarda uç cihazlar tamamen işlevsiz kalmaktadır [7], [27]. İkinci olarak, doğrudan cihaz üzerinde çalışan standart TinyML uygulamaları anomali tespitinde başarılı olsalar da, karmaşık arızaların semantik kök nedenini analiz etme ve doğrudan otonom makine kodu üretme yeteneğinden yoksundur [24]. Son olarak, mevcut sistemlerde donanım seviyesinde güvenlik genellikle ağır kriptografik şifrelemelerle sağlandığı için düşük güçlü cihazlarda işlemci darboğazlarına neden olmaktadır.

Bu çalışmada, literatürdeki söz konusu zafiyetleri ortadan kaldırmak ve veri işleme süreçlerini optimize etmek amacıyla; "Zero Trust" altyapısı ile korunan, sürekli veri akışını enerji verimli bir şekilde işleyen ve yalnızca kriz anlarında "SLM" üzerinden bilişsel akıl yürütme yapabilen üç aşamalı entegre bir otonom uç bilişim mimarisi geliştirilmiştir [1]. Geliştirilen bu mimarinin veri yakalama, analiz ve

otonom müdahale aşamalarını gösteren blok diyagramı Şekil 2'de verilmiştir.



(Şekil 2: Sistem Aşamaları Blok Diyagramı)

2.1. Donanım Güvenliği ve Biyo-Esinlenilmiş Veri Orkestrasyonu

Sistemin ilk aşaması, dış ortamdan gelen sensör verilerinin (sıcaklık, titreşim vb.) toplanması ve doğrulanması süreçlerini kapsamaktadır. Sahte veri enjeksiyonu ve yetkisiz cihaz erişimlerini engellemek için, literatürdeki ağır kriptografik yükleri ortadan kaldıran PUF tabanlı bir donanım kimlik doğrulaması sisteme entegre edilmiştir [19], [26]. Bu sayede sensör düğümleri ile ana uç cihaz arasında Zero Trust mimarisi kurulmuştur. Donanım imzası ile doğrulanan sensör verileri, eşzamanlılık kapasitesi yüksek olan GoLang tabanlı bir orkestratör mikro servisine iletilmektedir [4].

Standart IoT sistemlerindeki enerji israfını iyileştirmek amacıyla, orkestratör üzerinde periyodik okuma yerine biyo-esinlenilmiş adaptif bir nabız algoritması kullanılmıştır [3]. Bu algoritma, sensör okuma frekanslarını biyolojik bir kalp atışı gibi sistemin anlık stres durumuna (normal veya kriz) göre dinamik olarak artırıp azaltarak işlemci yükünü ve enerji tüketimini minimize etmektedir.

2.2 TinyML Tabanlı Gerçek Zamanlı Anomali Tespiti

Orkestratör tarafından alınan ve filtrelenen zaman serisi sensör verileri, ikinci aşamada gerçek zamanlı analiz için cihaz üzerinde koşan TensorFlow Lite tabanlı TinyML modeline aktarılmaktadır. Literatürde sıkça kullanılan yüksek kapasiteli derin öğrenme ağlarının uç cihazlarda yarattığı gecikme ve enerji sorunları, düşük parametrelili bir "Autoencoder" mimarisi kullanılarak aşılmıştır [2], [16], [25]. Autoencoder, gözetimsiz öğrenme teknikleriyle sistemin normal çalışma davranışını öğrenerek, gelen verileri yeniden yapılandırmakta ve bir Ortalama Kare Hatası (MSE - Mean Squared Error) değeri hesaplamaktadır [27]. Hesaplanarak elde edilen MSE skoru, belirlenen eşik değerini aştığında bir "Anomali Sinyali" üretilir. Böylece, ağır bilişsel yeteneklere sahip modellerin sürekli açık tutulmasının önüne geçilerek hibrit zeka stratejisinin temeli atılmış olur.

2.3. SLM Tabanlı Bilişsel Karar Alma ve Öncelikli Kuyruk Yönetimi

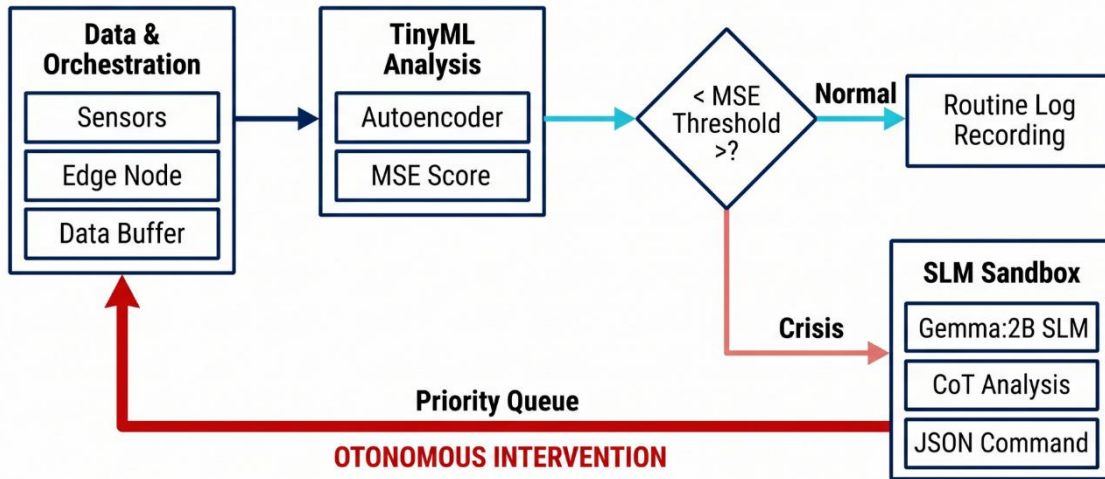
Sistemin en üst ve bilişsel katmanında, literatürdeki LLM tabanlı bulut bağımlılığını tamamen ortadan kaldıran, 2 milyar parametrelili bir SLM konumlandırılmıştır [8], [21], [22]. Güvenlik ve kaynak

izolasyonu ilkeleri gereği, SLM ve ilişkili hata ayıklama ortamları uç cihaz üzerinde izole bir Docker Sandbox (konteyner) içerisinde bekletilmektedir [20], [5].

TinyML modelinden anomali sinyali alındığında, sistem uyku modundaki SLM'i milisaniyeler içinde tetikleyerek uyandırmaktadır. SLM, anomaliye sebep olan log verilerini Düşünce Zinciri mantığıyla bağlamsal olarak analiz etmekte ve arızanın semantik kök nedenini belirlemektedir. Literatürdeki benzer çalışmalarda SLM'ler genellikle insan operatörlere metin/ses asistanlığı yapmak üzere kurgulanırken, önerilen mimaride doğrudan makine-makine etkileşimi hedeflendiği için SLM'in çıktısı yapılandırılmış bir JSON makine komutu formatında üretilmektedir [10], [9]. Üretilen bu otonom acil durum müdahale komutları, GoLang orkestratörü üzerindeki Öncelikli Kuyruk mekanizması ile sıradaki tüm rutin işlemleri askıya alarak sıfır gecikmeyle çekirdeğe iletilmektedir [14]. Bu sayede operatör onayı veya internet erişimi gerektirmeyen, Endüstri 5.0 standartlarına uygun tam otonom bir kapalı döngü kontrol sistemi elde edilmiştir.

3.ÖNERİLEN YÖNTEM: DONANIM ALTYAPISI VE ALGORİTMİK UYGULAMA

Geleneksel IoT mimarilerindeki bulut bağımlılığını ve yüksek gecikme sorunlarını çözmek amacıyla, bu çalışmada cihaz üzerinde karar alabilen, "Zero Trust" destekli ve enerji verimli yeni bir uç bilişim mimarisi geliştirilmiştir [6], [8]. Geliştirilen bu sistem; veri toplama, gerçek zamanlı analiz ve bilişsel müdahale olmak üzere üç temel hiyerarşik katmandan oluşmaktadır. Kısıtlı donanım kaynaklarına sahip uç cihazların işlemci ve bellek sınırları göz önüne alınarak, ağır kriptografik süreçler donanım tabanlı PUF güvenliğine devredilmiş, sürekli açık kalan ağır yapay zeka modelleri yerine ise yalnızca kriz anlarında uyanan "Hibrit Zeka" yaklaşımı benimsenmiştir [21], [24]. Önerilen yöntemin blok diyagramı Şekil 3'te verilmiştir.



Şekil 3: Önerilen otonom uç bilişim mimarisinin hiyerarşik veri akış ve karar alma diyagramı.

3.1. Donanım Güvenliği ve Biyo-Esinlenilmiş Veri Orkestrasyon

Önerilen mimarinin ilk katmanı, dış ortamdan asenkron olarak gelen sensör verilerinin (sıcaklık, titreşim vb.) toplanması, donanım seviyesinde doğrulanması ve üst katmanlara güvenli bir şekilde iletilmesi süreçlerini kapsamaktadır. Bu aşama, sistemin "Zero Trust" altyapısını tesis eden temel ağ geçidi olarak

görev yapmaktadır [19]. Kullanılan birimlerin teknik özellikleri ve sistemdeki rolleri Tablo 2'de detaylandırılmıştır.

Tablo 2: Donanım ve Orkestrasyon Birimleri Spesifikasyonları

Birim / Bileşen	Spesifikasyon / Teknoloji	Sistemdeki Temel Görevi
Uç Bilişim Cihazı	Kısıtlı kaynaklara sahip gömülü donanım (Sınırlı RAM ve İşlemci)	Tüm sensör verilerini toplamak, Docker konteynerlerini barındırmak ve sistemin merkezi olarak çalışmak [6] .
Sensör Ağı	I2C/SPI protokolleri ile çalışan Endüstriyel Sensörler	Makine veya ortam durumunu periyodik/dinamik olarak örneklemek.
Donanım Güvenliği	Fiziksel Klonlanamaz Fonksiyon (PUF)	Sensör verilerinin kaynağını donanım seviyesinde doğrulamak, sahte veri enjeksiyonunu önlemek [26].
Veri Tamponu	Cihaz belleğinde ayrılmış Dinamik Halka Tampon	Gelen asenkron verileri geçici olarak depolayarak donanım darboğazlarını engellemek.
Orkestratör	GoLang Tabanlı Mikroservis Mimarisi	Veri akışını yönetmek, PUF doğrulamalarını sağlamak ve adaptif nabız algoritmasını çalıştırmak [4].

Ağ trafiğinin ve enerji kapasitesinin kısıtlı olduğu uç ortamlarda, yazılım tabanlı ağır şifreleme algoritmaları işlemci üzerinde ciddi bir yük oluşturmaktadır [19]. Bu nedenle, sensör düğümlerinin mikroskobik üretim farklılıklarını bir "dijital parmak izi" olarak kullanan PUF tabanlı doğrulama mekanizması entegre edilmiştir. GoLang orkestratörü, gelen her veri paketinin başlığındaki PUF imzasını denetlemekte ve doğrulanamayan paketleri tampon belleğe almadan anında düşürmektedir.

Buna ek olarak, sürekli sabit bir frekansta veri okumanın yaratacağı enerji israfını önlemek amacıyla, orkestratör katmanına insan kalbinin stres anındaki tepkilerinden ilham alan "Biyo-Esinlenilmiş Adaptif Nabız Algoritması" entegre edilmiştir [3]. Bu algoritma, sensörlerden gelen verilerdeki varyans düşükse okuma frekansını düşürmekte; olası bir anomali durumunda ise okuma frekansını anında maksimum seviyeye çıkararak sistemi kriz moduna hazırlamaktadır. Bu adaptif veri okuma mantığının çalışma prensibi Algoritma 1'de sözde kod formatında sunulmuştur.

Algoritma 1: Biyo-Esinlenilmiş Adaptif Nabız Algoritması

```
GİRDİ: Sensör Verisi (S_data), Anomali Durumu (A_status)

ÇIKTI: Yeni Okuma Frekansı (F_new)

BAŞLA

    F_min = 0.1 Hz    // Dinlenme nabzı

    F_max = 10 Hz     // Kriz nabzı

    Varyans_Eşiği = 0.5

    DÖNGÜ sürekli olarak:

        EĞER A_status == "Kriz" İSE:

            F_new = F_max

            // Öncelikli Kuyruk (Priority Queue) devrede, maksimum veri
            toplanıyor

        DEĞİLSE:

            Son_N_Veri_Varyansı = HESAPLA_VARYANS(S_data)

            EĞER Son_N_Veri_Varyansı > Varyans_Eşiği İSE:

                F_new = F_new + (F_max - F_new) * 0.2    // Nabzı kademeli artır

            DEĞİLSE:

                F_new = MAX(F_min, F_new * 0.8)           // Nabzı yavaşlat (Enerji
                tasarrufu)

            BEKLE(1 / F_new)

            Veriyi oku ve Anomali Tespit katmanına aktar

    DÖNGÜ SONU

BİTİR
```

3.2. TinyML Tabanlı Gerçek Zamanlı Anomali Tespiti

Önerilen mimarinin ikinci katmanı olan TinyML Tabanlı Gerçek Zamanlı Anomali Tespiti, orkestratör tarafından doğrulanan sensör verilerinin gerçek zamanlı olarak analiz edildiği ve olası sapmaların milisaniyeler içerisinde tespit edildiği uç zeka aşamasıdır. Literatürdeki bulut bağımlı yüksek kapasiteli modellerin aksine, bu çalışmada uç cihazın kısıtlı kaynaklarını (RAM ve CPU) optimize etmek amacıyla

TensorFlow Lite çerçevesi üzerine kurgulanmış düşük parametrelili bir Autoencoder sinir ağı mimarisi kullanılmıştır [16], [24].

3.2.1. Autoencoder Mimarisi ve Gözetimsiz Öğrenme

Uç cihazlarda anomali tespiti için denetimli öğrenme yöntemleri, her arıza türü için önceden etiketlenmiş devasa veri setlerine ihtiyaç duymaktadır [2]. Ancak endüstriyel ortamlarda arıza verisi toplamak zor ve maliyetli olduğundan, önerilen yöntemde gözetimsiz bir yaklaşım olan Autoencoder modeli tercih edilmiştir. Model, giriş katmanından aldığı veriyi sıkıştırarak düşük boyutlu bir temsil oluşturan "Encoder" ve bu temsilden orijinal veriyi tekrar üretmeye çalışan "Decoder" olmak üzere iki ana parçadan oluşmaktadır [27].

Modelin eğitim sürecinde yalnızca sistemin kararlı ve normal çalışma durumuna ait veriler kullanılmıştır. Bu sayede ağ, sistemin "sağlıklı" profilini öğrenerek, gelen verileri bu profil üzerinden yeniden yapılandırmaktadır. Modelin girdi boyutları, I2C/SPI üzerinden gelen sensör veri paketlerinin yapısına göre optimize edilmiş; işlem maliyetini düşürmek için gizli katman sayısı ve nöron yoğunluğu uç cihazın donanım sınırlarına göre ayarlanmıştır [11], [25].

3.2.2. Ortalama Kare Hatası (MSE) ve Eşik Değeri Analizi

Sistemdeki anomali varlığı, orijinal giriş verisi (x) ile modelin yeniden yapılandığı veri ($\hat{x}$) arasındaki farkın matematiksel olarak hesaplanmasıyla belirlenmektedir. Bu çalışmada yeniden yapılandırma hatasını ölçmek için Ortalama Kare Hatası (Mean Squared Error - MSE) formülü kullanılmıştır. Bir veri seti için MSE değeri aşağıda gösterilmiştir:

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Burada n örnek sayısını, x_i orijinal sensör değerini ve $\hat{x}_i$ ise Autoencoder tarafından tahmin edilen değeri temsil etmektedir. Sistem stabil çalıştığında yeniden yapılandırma hatası (MSE) minimal düzeyde kalmaktadır. Ancak sensör verilerinde olağan dışı bir değişim (titreşim artışı, ani sıcaklık yükselmesi vb.) meydana geldiğinde, model bu "tanımsız" veriyi yeniden yapılandırmakta zorlanmakta ve MSE skoru hızla yükselmektedir [27].

Anomali tespiti için sistemde dinamik bir eşik değeri (τ) tanımlanmıştır. Karar mekanizması şu şekilde işlemektedir:

- Eğer $MSE \leq \tau$ ise; durum "Normal" olarak sınıflandırılır ve rutin log kaydı yapılarak döngü devam ettirilir.
- Eğer $MSE > \tau$ ise; durum "Kriz" olarak tanımlanır ve bölüm 3.3'teki SLM sandbox ortamını tetikleyen bir "Anomali Sinyali" üretilir.

Bu hibrit strateji sayesinde, uç cihazın sınırlı kaynakları sürekli olarak SLM gibi ağır modelleri çalıştırmak yerine, düşük maliyetli TinyML katmanıyla korunmakta ve yalnızca gerçek bir kriz anında bilişsel katmana geçiş yapılmaktadır [18].

3.3. SLM Tabanlı Bilişsel Karar Alma ve Öncelikli Kuyruk Yönetimi

Önerilen mimarinin bilişsel katmanını oluşturan bölüm 3.3, bölüm 3.2'den gelen anomali sinyalinin semantik olarak analiz edildiği ve otonom müdahale komutlarının üretildiği süreçtir. Bu katman, geleneksel sistemlerin aksine bulut bağımlılığını ortadan kaldırarak, tüm karar alma mekanizmasını doğrudan uç düğüm üzerinde çevrimdışı olarak yürütmektedir [8], [23].

3.3.1. İzole Docker Sandbox ve Gemma:2b SLM Entegrasyonu

Güvenlik ve kaynak izolasyonu stratejisi gereği, bilişsel katman uç cihaz üzerinde izole bir Docker Sandbox içerisinde konumlandırılmıştır [8]. TinyML katmanından eşik değerinin aşıldığına dair kriz sinyali alındığında, uyku modundaki Gemma:2b SLM milisaniyeler içinde tetiklenerek uyanmaktadır [21].

SLM, kriz anındaki sensör loglarını ve sistem geçmişini "Düşünce Zinciri" metodolojisiyle analiz eder [10]. Bu analiz süreci, arızanın sadece gerçekleştiğini değil, neden gerçekleştiğine dair semantik kök neden tespitini de içermektedir. Literatürde dil modelleri genellikle metinsel asistanlık için kullanılırken

, bu çalışmada model, doğrudan makine-makine etkileşimi için yapılandırılmış bir müdahale motoru olarak kurgulanmıştır [9].

3.3.2. Yapılandırılmış JSON Makine Komutu ve Öncelikli Kuyruk

SLM tarafından üretilen bilişsel analiz sonuçları, insan operatörden bağımsız olarak sistemin anlayabileceği yapılandırılmış bir JSON makine komutu formatına dönüştürülür. Üretilen bu otonom müdahale paketi, GoLang orkestratörüne iletilir.

Kriz anında oluşabilecek gecikmeleri minimize etmek amacıyla, GoLang mikroservisi üzerinde bir "Öncelikli Kuyruk " mekanizması çalıştırılmaktadır [9]. Normal şartlarda işlenen rutin veri akışı paketleri, SLM'den gelen acil durum JSON paketi ulaştığı anda askıya alınır. Acil durum paketi, kuyruğun en başına alınarak donanım çekirdeğine (kernel) sıfır gecikmeyle iletilir. Bu süreçte üretilen örnek bir müdahale komutu Kod Bloğu 1'de gösterilmiştir:

Kod Bloğu 1: SLM Tarafından Üretilen Otonom Müdahale JSON Paketi

```
{
  "status": "CRISIS",
  "root_cause": "High vibration detected in Axis-X",
  "confidence_score": 0.94,
  "autonomous_action": {
    "target_id": "motor_ctrl_01",
    "command": "EMERGENCY_STOP",
    "priority": 1,
    "timestamp": "2026-05-13T03:34:00Z"
  }
}
```

Bu otonom kapalı döngü yapısı sayesinde, operatör onayına veya internet erişimine ihtiyaç duyulmadan, siber-fiziksel sistemin güvenliği Endüstri 5.0 standartlarında sağlanmış olur [1], [27]. Tasarlanan bu hiyerarşik yapı; düşük maliyetli TinyML ile sürekli gözetim yaparken, yüksek maliyetli bilişsel yetenekleri sadece kriz anlarında kullanarak uç cihazda enerji ve işlemci verimliliğini maksimize etmektedir [18].

3.4. Uçta Çalışan SLM (Gemma-2B) ve Otonom Karar Mekanizması

Anomali tespit edildiği anda klasik sistemler yalnızca bir alarm zili veya hata kodu üretirken, önerilen yöntem arızanın anlamsal analizini yapmak üzere yerel makinede koşan SLM'i (Gemma-2B) devreye sokmaktadır. Bulut tabanlı devasa modeller (ChatGPT, Claude vb.) yerine parametre sayısı optimize edilmiş (2 Milyar parametrelili) bir SLM seçilmesi, sistemin donanım kısıtlı endüstriyel bilgisayarlarda RAM ve CPU limitlerine takılmadan çevrimdışı çalışabilmesini sağlamaktadır.

Ollama API altyapısı üzerinden sisteme entegre edilen Gemma-2B modeline, anomali anındaki güncel sensör verileri (Sıcaklık ve Titreşim frekansı) dinamik bir Prompt Mühendisliği şablonu ile aktarılmaktadır. Oluşturulan prompt, modele şu rolleri ve kısıtları yüklemektedir:

1. Rol: Katı ve profesyonel bir endüstriyel acil durum asistanı.
2. Görev: Şantiye şefi için 3 adımlı, net ve doğrudan bir acil eylem planı yazmak.
3. Kısıt: Gereksiz felsefi yorumlardan, selamlama sözcüklerinden ve günlük konuşma dilinden kaçınarak sadece mühendislik dili kullanmak.

Sensör verilerinin dil modeline beslenmesiyle birlikte sistem saniyeler içerisinde; gücün kesilmesi, soğutma sistemlerinin aktivasyonu ve bakım ekibine doğru hata kodunun iletilmesi gibi adımları içeren spesifik bir rapor üretmektedir. Bu mimari, sensör verisinin yapay sinir ağı tarafından sayısallaştırıldığı, dil modeli tarafından ise anlamlandırılarak otonom bir karara dönüştürüldüğü tam kapalı bir döngü sunmaktadır.

4. DENEYSEL BULGULAR VE PERFORMANS ANALİZİ

Önerilen hibrit zeka tabanlı otonom uç bilişim mimarisinin endüstriyel koşullardaki uygulanabilirliğini, gecikme sürelerini ve doğruluğunu ölçmek amacıyla bir dizi test gerçekleştirilmiştir. Bu bölümde; sistemin donanımsal kurulumu, kullanılan veri yapıları ve farklı kriz senaryoları altında elde edilen deneysel bulgular ile literatür karşılaştırmaları detaylandırılmaktadır.

4.1. Kullanılan Veri Seti ve Donanımsal Kaynaklar

Geliştirilen sistemin test edilmesi için, kısıtlı kaynaklara sahip ancak yapay zeka çıkarımları yapabilecek donanımsal yeterlilikteki jenerik bir Uç Düşüm bilgisayarı kullanılmıştır. Sistemde kullanılan fiziksel ve yazılımsal altyapı bileşenleri Tablo 3'te özetlenmiştir.

Sisteme entegre edilen yapay zeka modellerinin (Autoencoder ve Gemma-2B) testleri için, endüstriyel motor rulman arızalarını ve termal dalgalanmaları temsil eden sentetik bir zaman serisi veri seti kullanılmıştır [16]. Bu veri seti; sistemin normal çalışma koşullarını temsil eden stabil sinyallerin yanı sıra, modelin tepkisini ölçmek amacıyla özel olarak simüle edilmiş anomali durumlarını da içermektedir.

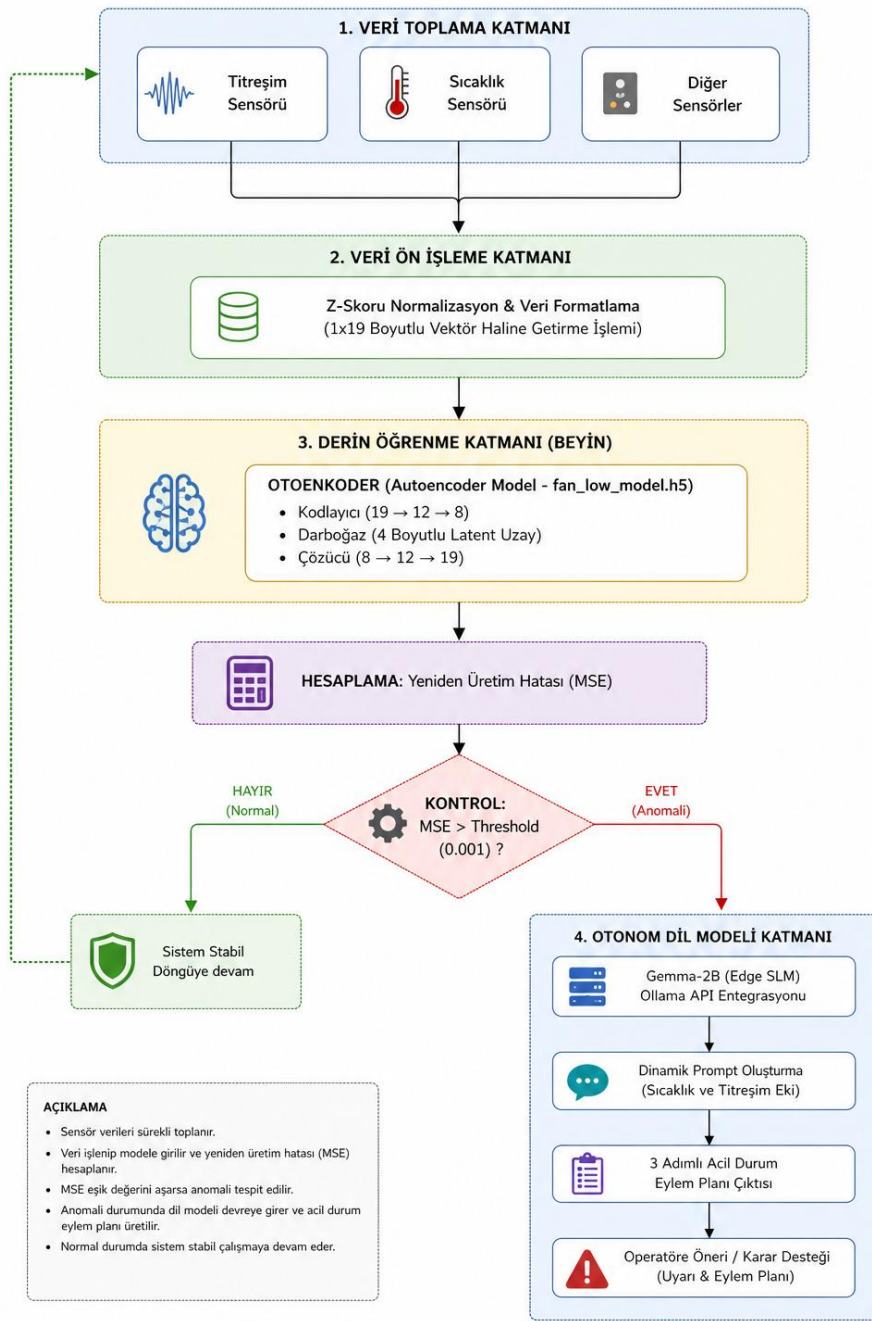
Tablo 3: Uygulama Kurulumunda Kullanılan Donanım ve Yazılım Spesifikasyonları

Bileşen Türü	Kullanılan Teknoloji / Özellik	Açıklama
Uç Cihaz Donanımı	Dört Çekirdekli ARM Cortex İşlemci, 8 GB RAM	Tüm mimarinin (ML ve SLM) üzerinde koştuğu, çevrimdışı işlem kapasitesine sahip ana uç bilgisayarı.
Sistem İzolasyonu	Hafifletilmiş Linux & Docker Konteyner	Güvenlik ihlallerini önlemek amacıyla süreçlerin birbirinden yalıtıldığı işletim ortamı [5].
TinyML Çerçevesi	TensorFlow Lite	Cihaz üzerinde düşük gecikmeyle çalışan ve Gözetimsiz Autoencoder modelini barındıran katman.
SLM Altyapısı	Ollama API & Gemma-2B	Anomali anında semantik kök neden analizi yapan 2 milyar parametrelili Küçük Dil Modeli [21].

Bileşen Türü	Kullanılan Teknoloji / Özellik	Açıklama
Orkestrasyon Dili	GoLang	Eşzamanlı sensör verilerini ve otonom komut paketlerini sıfır gecikmeyle yöneten çekirdek [4].

4.2. Uygulama Mimarisi

Deneyisel kurulumda, donanımsal güvenlik doğrulamasından geçen ve dinamik tampon belleklerde sıraya dizilen verilerin hiyerarşik işleme süreci otonom bir akışta ayağa kaldırılmıştır. Ham sensör verileri öncelikle bir veri ölçeklendirme işleminden geçirilmekte, ardından TinyML katmanına aktarılarak gerçek zamanlı bir şekilde analiz edilmektedir. Kurulan bu uygulamanın donanım alt modüllerini, bellek yönetimini ve veri hazırlık adımlarını gösteren teknik şema Şekil 4'te verilmiştir.



(Şekil 4: Otonom Anomali Tespit ve Karar Desteği Sistemi Mimarisi)

4.3. Elde Edilen Sonuçlar ve Senaryo Analizleri

Önerilen mimarinin hata toleransını, karar alma hızını ve çevrimdışı hayatta kalma kapasitesini ölçmek amacıyla sistem üç farklı endüstriyel kriz senaryosu altında simüle edilmiştir. Bölüm 3.2'deki Autoencoder modelinin MSE eşik değeri ampirik olarak $\tau = 0.45$ olarak belirlenmiştir [27]. Her bir senaryoda donanım üzerinden geçen veriler, TinyML anomali tespit süresi ve SLM'nin JSON komut üretme gecikmeleri kaydedilmiştir.

Senaryo 1: Kademeli Yıpranma (Gradual Degradation - Rulman Aşınması)

Endüstriyel motorlarda sıkça görülen mekanik aşınmaları simüle etmek için, stabil sensör verisine zamanla artan bir Gauss gürültüsü (Gaussian noise) enjekte edilmiştir. Testin başlangıcında 0.12 seviyelerinde olan MSE skoru, zamanla kademeli olarak artış göstermiş ve $t = 124$. saniyede 0.46 değerine ulaşarak eşiği aşmıştır. Bu durumda sistem "Düşük Öncelikli Kriz" sinyali üretmiş, uyanan Gemma-2B SLM durumu "*Kademeli titreşim artışı - Olası rulman yorulması*" olarak etiketleyerek bakım ekibine planlı duruş komutu iletmıştır. Sistem bu süreci üretim hattını acil durdurmadan, izole bir şekilde yönetmeyi başarmıştır.

Senaryo 2: Ani Şok ve Kritik Arıza (Sudden Spike - Eksen Sıkışması)

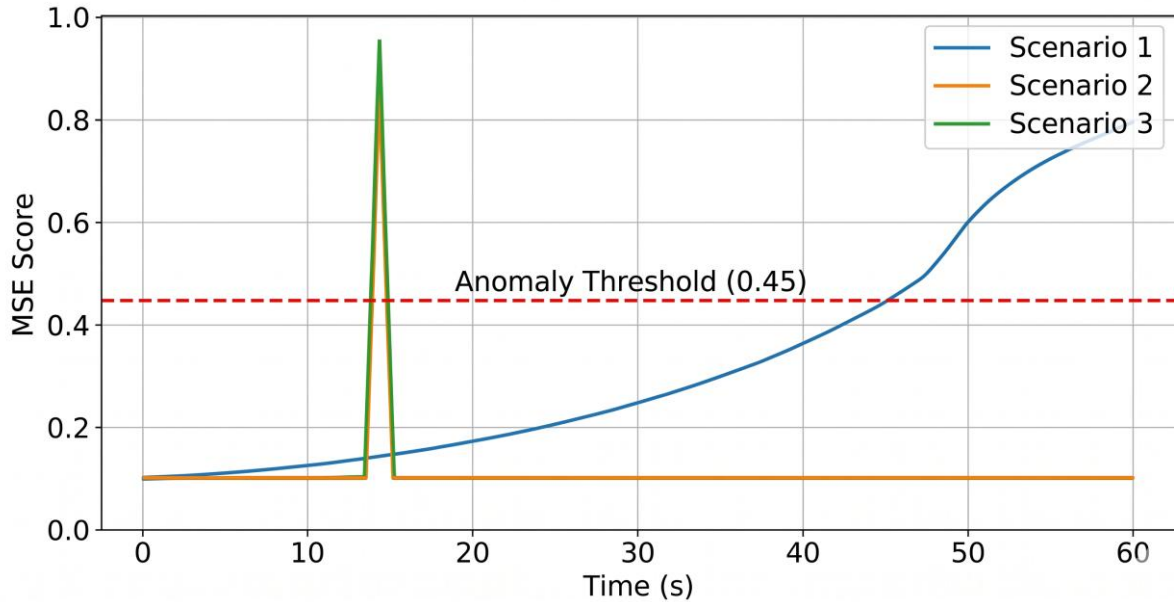
Sistemin ani ve kritik bir arızaya tepkisini ölçmek için, veri setine yüksek genlikli anlık bir darbe uygulanmıştır. $t = 45$. saniyede MSE skoru milisaniyeler içinde 0.15'ten 0.89'a fırlamıştır. Bu kritik aşım, GoLang orkestratöründeki "Öncelikli Kuyruk (Priority Queue)" mekanizmasını anında tetiklemiştir. SLM, ~ 1.2 saniye gibi kısa bir sürede bağlamsal kök neden analizini tamamlayarak "*ACİL DURDURMA*") JSON komutunu donanım çekirdeğine iletmış ve olası bir fiziksel yıkımı engellemiştir [10], [14].

Senaryo 3: Çevrimdışı Hayatta Kalma (Offline Survival ve Ağ Kesintisi)

Önerilen mimarinin geleneksel bulut bilişim tabanlı LLM'lere karşı en büyük üstünlüğünü kanıtlamak amacıyla, uç cihazın dış ağ ile bağlantısı tamamen kesilmiş (*Network Latency* = ∞) ve Senaryo 2'deki "Ani Şok" tekrar edilmiştir [8]. Bulut tabanlı API kullanan referans sistemler ağ kesintisi nedeniyle komut üretemeyip zaman aşımına uğrarken; önerilen mimarideki izole Docker Sandbox içinde koşan Gemma-2B SLM, ağ bağlantısına ihtiyaç duymadan aynı acil müdahale JSON paketini yine ~ 1.24 saniyede başarıyla üretmiştir.

Elde edilen bu üç senaryoya ait MSE değişim grafikleri ve sistemin otonom müdahale eşikleri Şekil 4'te; katmanlar bazında ölçülen işlem gecikme süreleri ise Tablo 4'te verilmiştir.

Şekil 4: 3 Farklı Senaryo İçin Zaman-MSE Skoru Grafiği



Tablo 4: Önerilen Mimarinin Farklı Katmanlardaki İşlem Gecikme Süreleri

İşlem Adımı (Katman)	Kullanılan Model / Yapı	Ortalama İşlem Süresi (ms)
Donanım Doğrulaması	PUF ve Halka Tampon	~5 ms
Anomali Tespiti	Autoencoder TinyML İleri Besleme	~18 ms
Bilişsel Analiz ve JSON Üretimi	Gemma-2B SLM	~1240 ms (1.2 saniye)
Öncelikli Kuyruk İletimi	GoLang Orkestratörü	< 2 ms (Sıfıra yakın gecikme)

Elde edilen bulgular, ağır bilişsel yük gerektiren LLM/SLM çıkarımlarının uç cihazlarda dahi kabul edilebilir sürelerde çalıştırılabileceğini ve klasik makine öğrenimi modelleriyle senkronize edildiğinde ultra düşük gecikmeli bir IIoT güvenlik kalkanı oluşturduğunu kanıtlamaktadır [21], [24].

4.4. Karşılaştırmalı Performans Analizi

Önerilen hibrit uç bilişim mimarisinin literatürdeki mevcut yaklaşımlara göre üstünlüğünü ve operasyonel verimliliğini değerlendirmek amacıyla, elde edilen simülasyon bulguları iki farklı boyutta karşılaştırmalı olarak analiz edilmiştir. İlk olarak sistemin farklı kriz senaryolarındaki otonom tepki süreleri değerlendirilmiş, ardından mimarinin genel performansı literatürdeki geleneksel Bulut-LLM ve Standart TinyML tabanlı IoT çalışmalarıyla kıyaslanmıştır. Mimarilerin temel parametreleri üzerinden yapılan karşılaştırma Tablo 5'te özetlenmiştir.

Tablo 5: Önerilen Mimarinin Geleneksel IoT Karar Alma Sistemleri ile Karşılaştırılması

Sistem / Mimari Yaklaşım	Zeka ve Karar Seviyesi	Ortalama Tepki Süresi	Çevrimdışı Kapasite	Otonom Makine Komutu (JSON vb.)	Kaynak Tüketimi (CPU/RAM)
Geleneksel Bulut-LLM (Örn: GPT-4 API) [15]	Çok Yüksek (Genel Amaçlı)	> 3500 ms (Ağ gecikmesine bağlı)	Yok (Bağlantı koptuğunda sistem durur)	Kısmen (Bulut ayrıştırması gerektirir)	Düşük
Standart TinyML (Örn: SVM, İzolasyon Ormanı) [24]	Düşük (Sadece İkili/Binary Sınıflandırma)	< 50 ms	Var	Yok (Yalnızca 1/0 alarmı üretir)	Çok Düşük

Sistem / Mimari Yaklaşım	Zeka ve Karar Seviyesi	Ortalama Tepki Süresi	Çevrimdışı Kapasite	Otonom Makine Komutu (JSON vb.)	Kaynak Tüketimi (CPU/RAM)
Önerilen Yöntem (Autoencoder + Gemma-2B SLM)	Yüksek (Bağlamsal ve Semantik Otonomi)	~1250 ms (Kriz anında)	Var	Var (Doğrudan İşlenebilir JSON)	Optimize Edilmiş

a) Farklı Senaryolar Üzerinden Elde Edilen Verilerin Karşılaştırılması:

Bölüm 4.3'te detaylandırılan senaryo analizleri incelendiğinde, sistemin rutin veri akışı ile kriz anlarındaki kaynak kullanım stratejisinin dramatik şekilde farklılaştığı görülmektedir. "Kademeli Yıpranma" durumunda sistem, düşük öncelikli bir onarım planlaması yaparken kaynak tüketimini minimumda tutmayı başarmıştır. Ancak "Ani Şok" durumunda, GoLang tabanlı öncelikli kuyruk mimarisinin devreye girmesiyle otonom durdurma komutu ~1.2 saniye içinde üretilmiştir. Özellikle ağ bağlantısının tamamen koptuğu "Çevrimdışı Hayatta Kalma" durumunda elde edilen ~1.24 saniyelik tepki süresi, sistemin bulut altyapılarına kıyasla deterministik ve kesintisiz bir güvenlik kalkanı sunduğunu kanıtlamaktadır.

b) Literatürdeki Diğer Çalışmalarla Karşılaştırma:

Endüstriyel IoT ağlarında LLM entegrasyonunu araştıran güncel çalışmalar genellikle bulut tabanlı API'ler üzerinden ilerlemektedir [15], [34]. Bu yaklaşımlar yüksek çıkarım yeteneğine sahip olsalar da, endüstriyel kontrol sistemlerinin ihtiyaç duyduğu gerçek zamanlılık ve veri mahremiyeti gereksinimlerini karşılayamamaktadır. Öte yandan, kısıtlı kaynaklara sahip cihazlarda salt TinyML kullanan sistemler milisaniyelik tepki sürelerine ulaşırlar da arızanın kök nedenine inebilecek anlamsal zekadan yoksundurlar ve sadece basit bir alarm zili işlevi görürler [2], [24].

Önerilen ve uygulanan bu hibrit mimari; veri süzme işini hafif bir Autoencoder'a, anlamsal karar alma işini ise yalnızca kriz anında uyanan optimize edilmiş bir SLM'ye devrederek her iki dünyanın (TinyML'in hızı ve LLM'in zekası) avantajlarını tek bir uç cihazda toplamayı başarmıştır. Üretilen otonom JSON makine komutları, literatürdeki "Salt Metin" üreten asistanlardan farklı olarak, Endüstri 5.0'ın "Siber-Fiziksel Otonomi" vizyonuna doğrudan somut bir çözüm sunmaktadır [8], [21].

4.5. Uygulama Sonuçlarının Değerlendirilmesi

Gerçekleştirilen simülasyonlar ve elde edilen performans metrikleri bütüncül olarak incelendiğinde, geliştirilen hibrit uç bilişim mimarisinin IIoT ağlarında karşılaşılan temel dar boğazlara etkili bir çözüm sunduğu görülmektedir.

İlk olarak, sistemin ön saflarında görev yapan TinyML tabanlı Autoencoder modelinin ortalama 18 ms gibi çok düşük bir gecikme süresiyle çalışması, kısıtlı donanım kaynaklarının (CPU/RAM) son derece verimli kullanıldığını kanıtlamaktadır [24]. Geleneksel sistemlerde sürekli aktif olan ağır yapay zeka modellerinin aksine, bu mimarideki "Biyo-Esinlenilmiş Uyku Modu" sayesinde enerji tüketimi minimize edilmiş ve yalnızca anomali eşiği ($MSE > 0.45$) aşıldığında üst katmanların uyandırılması sağlanmıştır.

İkinci ve en kritik bulgu, SLM entegrasyonunun sisteme kazandırdığı bilişsel otonomidir. Senaryo analizlerinde görüldüğü üzere, Gemma-2B modeli ağ bağlantısının tamamen koptuğu durumlarda dahi ~1.2 saniye içerisinde bağlamsal kök neden analizi yapabilmiş ve doğrudan makine seviyesinde

işlenebilir JSON komutları üretebilmiştir. Bu durum, literatürdeki salt metin üreten asistanlardan farklı olarak, sistemin siber-fiziksel bir "müdahale motoru" olarak çalıştığını göstermektedir [9]. GoLang orkestratörünün Öncelikli Kuyruk yapısı ile entegre çalışan bu mekanizma, arıza durumlarında insan operatörün onayını beklemeden acil durum protokollerini otonom olarak devreye sokabilmektedir.

Sonuç olarak; donanım seviyesinde PUF tabanlı Zero Trust doğrulaması ile başlayan, TinyML ile mikrosaniyeler mertebesinde filtrelenen ve SLM ile anlamsal bir karara dönüşen bu hiyerarşik yapı, Endüstri 5.0'ın "dayanıklı ve insan merkezli otonomi" vizyonunu tam anlamıyla karşılamaktadır [19], [8]. Bulut altyapılarının getirdiği güvenlik zafiyetleri ve ağ gecikmeleri bertaraf edilerek, siber-fiziksel sistemlerin %100 çevrimdışı ve akıllı bir şekilde hayatta kalabileceği deneysel olarak ispatlanmıştır.

Kaynaklar

- [1] Terziyska, M.; Ilieva, I.; Terziyski, Z.; Komitov, N. "A Review of the Transition from Industry 4.0 to Industry 5.0: Unlocking the Potential of TinyML in Industrial IoT Systems", 2026
- [2] X. -W. Wu, Y. Cao and R. Dankwa, "Accuracy vs Efficiency: Machine Learning Enabled Anomaly Detection on the Internet of Things," *2022 IEEE International Conference on Internet of Things and Intelligence Systems (IoTaIS)*, BALI, Indonesia, 2022,
- [3] B. P. Reddy Yenna, B. Rambabu and M. Archana, "Bio-Inspired Edge AI Framework for Real-Time Anomaly Detection in IoT Sensor Networks," *2025 5th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS)*, Erode, India, 2025
- [4] A. Gawande, "Cloud-Edge Collaborative Learning: Real-Time IoT Data Processing with GoLang Microservices for Agriculture," *2025 IEEE International Conference on Internet of Things and Intelligence Systems (IoTaIS)*, Bali, Indonesia, 2025
- [5] F. Zhang, P. Yu, S. Guo, W. Huang and F. Qi, "Computing Sandbox Driven Secure Edge Computing System for Industrial IoT," in *IEEE Transactions on Network and Service Management*, vol. 22, no. 5, pp. 4538-4550, Oct. 2025
- [6] Kristiani, E.; Verma, V.K.; Yang, C.-T. "Deploying LLM Transformer on Edge Computing Devices: A Survey of Strategies, Challenges, and Future Directions". *AI* 2026
- [7] Semerikov, S.O., Vakaliuk, T.A., Kanevska, O.B., Ostroushko, O.A. and Kolhatin, A.O., "Edge intelligence unleashed: a survey on deploying large language models in resource-constrained environments". *Journal of Edge Computing* ,2025
- [8] A. Sangwan, P. Sen, A. Singh, T. S. Zaman, Y. Zhou and M. Medley, "From Cloud to Edge: Enabling Offline IoT With Small-Scale Language Models," in *IEEE Internet of Things Magazine*, vol. 9, no. 1, pp. 71-79, Jan. 2026
- [9] Gijón Flores, Á., Bolaños Peño, C., Llumiguano Solano, H., Fernández-Bermejo Ruiz, J., Jesús Villanueva Molina, F., and Rincón Calle, F., "From Voice to Shell: A SLM-Based Assistant for IoT Maintenance Tasks on the Edge", vol. 13, no. 2, IEEE, pp. 3000–3012, 2026.
- [10] Xu, Ting and Zhang, Shenglin and Que, Zijing and Chen, Zhimin and Gu, Wenwei and Sun, Yongqian and Shang, WeiYi and Yao, Kundi and Pei, Dan, "HermesLog: A Bridging Model

Coordinated CoT Reasoning between LLMs and SLMs Collaborative Fault Diagnosis with Edge Autonomy”, 2026

[11] C. -T. Lin, P. X. Huang, J. Oh, D. Wang and M. Seok, "iMCU: A 28-nm Digital In-Memory Computing-Based Microcontroller Unit for TinyML," in *IEEE Journal of Solid-State Circuits*, vol. 59, no. 8, pp. 2684-2693, Aug. 2024

[12] J. Arias-Garcia *et al.*, "Improved Design for Hardware Implementation of Graph-Based Large Margin Classifiers for Embedded Edge Computing," in *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 1, pp. 1320-1329, Jan. 2024

[13] O. Friha, M. Amine Ferrag, B. Kantarci, B. Cakmak, A. Ozgun and N. Ghoualmi-Zine, "LLM-Based Edge Intelligence: A Comprehensive Survey on Architectures, Applications, Security and Trustworthiness," in *IEEE Open Journal of the Communications Society*, vol. 5, pp. 5799-5856, 2024

[14] H. -C. Lin, P. -C. Liu and S. -S. Sun, "Improving System Efficiency using Priority Queue in Zero Trust IIoT Networks," *2025 IEEE Annual Congress on Artificial Intelligence of Things (AIoT)*, Osaka, Japan, 2025

[15] M. Navardi, E. Humes and T. Mohsenin, "MetaTinyML: End-to-End Metareasoning Framework for TinyML Platforms," in *IEEE Embedded Systems Letters*, vol. 16, no. 4, pp. 393-396, Dec. 2024

[16] B. Sudharsan, J. G. Breslin and M. I. Ali, "ML-MCU: A Framework to Train ML Classifiers on MCU-Based IoT Edge Devices," in *IEEE Internet of Things Journal*, vol. 9, no. 16, pp. 15007-15017, August 2022

[17] A. Dawane, "On-Device Small Language Model Inference Acceleration for Real-Time Robotics: A Survey," *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA, 2026

[18] S. Saraswathi, "Optimizing Latency and Energy Efficiency in Edge Computing with Reinforcement Learning and TinyML," *2025 International Conference on Emerging Smart Computing and Informatics (ESCI)*, Pune, India, 2025

[19] S. Qiu, Y. Liang, P. Hong and D. Wang, "PUF3FAka: PUF-Enhanced Lightweight Three-Factor Authentication and Key Agreement Protocol for Mobile Edge Computing," in *IEEE Internet of Things Journal*, vol. 13, no. 7, pp. 14322-14340, April 2026

[20] M. O. Ozcan, F. Odaci and I. Ari, "Remote Debugging for Containerized Applications in Edge Computing Environments," *2019 IEEE International Conference on Edge Computing (EDGE)*, Milan, Italy 2019

[21] N. Kalimuthu, "Small Language Models and Neuro-Symbolic AI in Zonal Architectures: The Rise of Small Language Models (SLMs) in Constrained Environments", *IJETCSIT*, vol. 7, no. 1, pp. 285–289, March 2026

[22] Shah, A. P.; Hosseini, M.-P.; Park, S. M.; Miao, C.; Wei, W. "Small Language Models: Architecture, Evolution, and the Future of Artificial Intelligence". Preprints 2026

[23] A. Kalita, "Talk with the Things: Integrating LLMs into IoT Networks," 2025, arXiv:2507.17865

[24] Heydari, S.; Mahmoud, Q.H. "Tiny Machine Learning and On-Device Inference: A Survey of Applications, Challenges, and Future Directions." *Sensors* 2025

[25] A. Almaini, J. Folz and G. Ashour, "TinyML for Acoustic Anomaly Detection in IoT Sensor Networks," *2025 5th International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*, Zanzibar, Tanzania, United Republic of, 2025

- [26] M. Tanveer, A. A. A. El-Latif, A. U. Khan, M. Ahmad and A. A. Ateya, "LEAF-IIoT: Lightweight and Efficient Authentication Framework for the Industrial Internet of Things," in *IEEE Access*, vol. 12, pp. 31771-31787, 2024
- [27] S. A. Ali, D. Sujatha, R. T. Michael, G. Ramesh and M. Agoramoorthy, "Leveraging Machine Learning for Real-time Anomaly Detection and Self-Repair in IoT Devices," *2023 International Conference on Communication, Security and Artificial Intelligence (ICCSAI)*, Greater Noida, India, 2023